

Chapter 5. User Datagram Protocol

[Section 5.1. Overview](#)

[Section 5.2. DatagramPacket Class](#)

[Section 5.3. DatagramSocket Class](#)

[Section 5.4. Listening for UDP Packets](#)

[Section 5.5. Sending UDP packets](#)

[Section 5.6. User Datagram Protocol Example](#)

[Section 5.7. Building a UDP Client/Server](#)

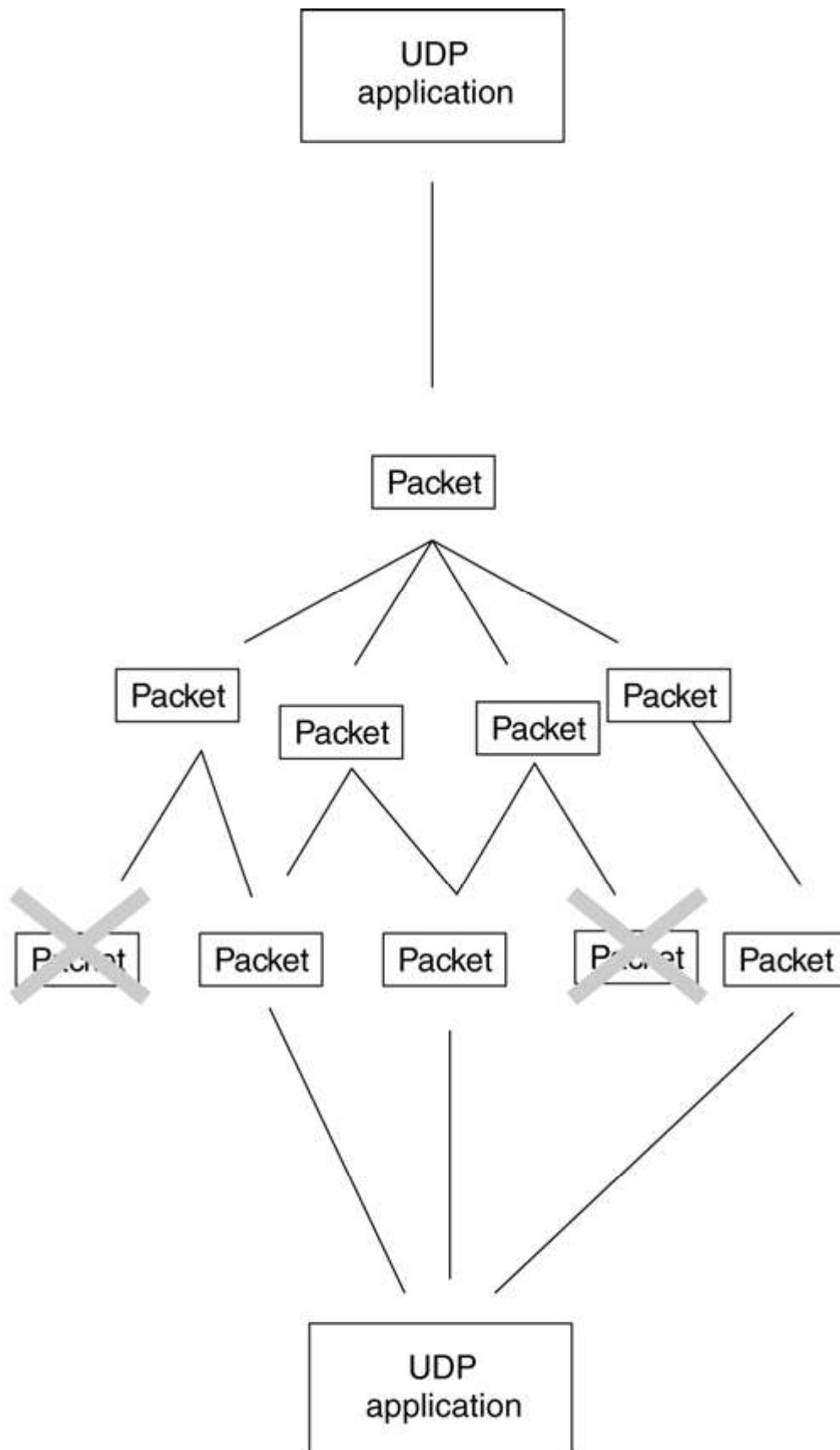
[Section 5.8. Additional Information on UDP](#)

[Section 5.9. Summary](#)

5.1 Overview

The User Datagram Protocol (UDP) is a commonly used transport protocol employed by many types of applications. UDP is a connectionless transport protocol, meaning that it doesn't guarantee either packet delivery or that packets arrive in sequential order. Rather than reading from, and writing to, an ordered sequence of bytes (using I/O streams, as discussed in [Chapter 4](#)), bytes of data are grouped together in discrete packets, which are sent over the network. Although control over the ultimate destination of a UDP packet rests with the computer that sends it, how it reaches that destination is an arbitrary process (see [Figure 5-1](#)).

Figure 5-1. UDP packet transport over a network can be unreliable.



The packets may travel along different paths, as selected by the various network routers that distribute traffic flow seemingly whimsically—depending on factors such as network congestion, priority of routes, and cost of transmission. (For example, for one packet a cheaper network route

might be selected, even though it is slower, but another might travel along a superfast pipeline if the cheaper alternative becomes too congested.) This means that a packet can arrive out of sequence, if it encounters a faster route than the previous packet (or if the previous packet encounters some other form of delay). No two packets are guaranteed the same route, and if a particular route is heavily congested, the packet may be discarded entirely. Each packet has a time-to-live (TTL) counter, which is updated when the packet is routed along to the next point in the network. When the timer expires, it will be discarded, and the recipient of the packet will not be notified. If a packet does arrive, however, it will always arrive intact. Packets that are corrupt or only partially delivered are discarded.

NOTE



Testing UDP applications in an intranet environment will yield various rates of packet loss (or none at all), due to the small number of hops a packet must make, and high bandwidth. Internet-based UDP transmission is more likely to result in dropped packets, and testing only in an intranet environment is dangerous, as developers may be unaware of the effects of lost packets.

Given the potential for loss of data packets, it may seem odd that anyone would even consider using such an unreliable, seemingly anarchical system. In fact, there are many advantages to using UDP that may not be apparent at first glance.

- UDP communication can be more efficient than guaranteed-delivery data streams. If the amount of data is small and the data is sent frequently (such as in the case of a counter whose previous value is irrelevant), it may make sense to avoid the overhead of guaranteed delivery.
- Unlike TCP streams, which establish a connection, UDP causes fewer overheads. If the amount of data being sent is small and the data is sent infrequently, the overhead of establishing a connection might not be worth it. UDP may be preferable in this case, particularly if data is being sent from a large number of machines to one central one, in which case the sum total of all these connections might cause significant overload.
- Real-time applications that demand up-to-the-second or better performance may be candidates for UDP, as there are fewer delays due to the error checking and flow control of TCP. UDP packets can be used to saturate available network bandwidth to deliver large amounts of data (such as streaming video/audio, or telemetry data for a multiplayer network game). In addition, if some data is lost, it can be replaced by the next set of packets with updated information, eliminating the need to resend old data that is now out of date.
- UDP sockets can receive data from more than one host machine. If several machines must be communicated with, then UDP may be more convenient than other mechanisms such as TCP (discussed in [Chapter 6](#)).
- Some network protocols specify UDP as the transport mechanism, requiring its use.

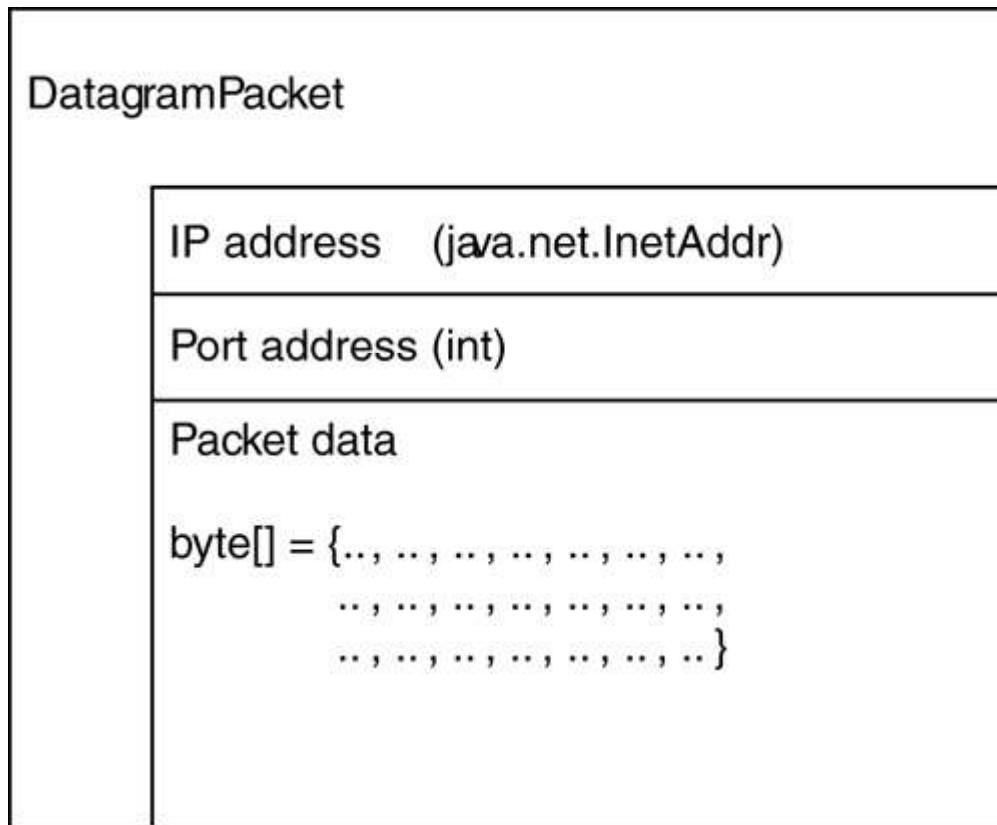
Java supports the User Datagram Protocol in the form of two classes:

- `java.net.DatagramPacket`
- `java.net.DatagramSocket`

5.2 DatagramPacket Class

The `DatagramPacket` class represents a data packet intended for transmission using the User Datagram Protocol (see [Figure 5-2](#)). Packets are containers for a small sequence of bytes, and include addressing information such as an IP address and a port.

Figure 5-2. `DatagramPacket` representation of a UDP packet



The meaning of the data stored in a `DatagramPacket` is determined by its context. When a `DatagramPacket` has been read from a UDP socket, the IP address of the packet represents the address of the sender (likewise with the port number). However, when a `DatagramPacket` is used to send a UDP packet, the IP address stored in `DatagramPacket` represents the address of the recipient (likewise with the port number). This reversal of meaning is important to remember—one wouldn't want to send a packet back to oneself!

5.2.1 Creating a `DatagramPacket`

There are two reasons to create a new `DatagramPacket`:

1. To send data to a remote machine using UDP
2. To receive data sent by a remote machine using UDP

Constructors

The choice of which `DatagramPacket` constructor to use is determined by its intended purpose. Either constructor requires the specification of a byte array, which will be used to store the UDP packet contents, and the length of the data packet.

To create a `DatagramPacket` for receiving incoming UDP packets, the following constructor should be used: `DatagramPacket(byte[] buffer, int length)`. For example:

```
DatagramPacket packet = new DatagramPacket(new byte[256], 256);
```

To send a `DatagramPacket` to a remote machine, it is preferable to use the following constructor: `DatagramPacket(byte[] buffer, int length, InetAddress dest_addr, int dest_port)`. For example:

```
InetAddress addr = InetAddress.getByName("192.168.0.1");
DatagramPacket packet = new DatagramPacket ( new byte[128],
128, addr, 2000);
```

5.2.2 Using a DatagramPacket

The `DatagramPacket` class provides some important methods that allow the remote address, remote port, data (as a byte array), and length of the packet to be retrieved. As of JDK1.1, there are also methods to modify these, via a corresponding set method. This means that a received packet can be reused. For example, a packet's contents can be replaced and then sent back to the sender. This saves having to reset addressing information—the address and port of the packet are already set to those of the sender.

Methods

- `InetAddress getAddress()`—returns the IP address from which a `DatagramPacket` was sent, or (if the packet is going to be sent to a remote machine), the destination IP address.
- `byte[] getData()`—returns the contents of the `DatagramPacket`, represented as an array of bytes.
- `int getLength()`—returns the length of the data stored in a `DatagramPacket`. This can be less than the actual size of the data buffer.
- `int getPort()`—returns the port number from which a `DatagramPacket` was sent, or (if the packet is going to be sent to a remote machine), the destination port number.
- `void setAddress(InetAddress addr)`—assigns a new destination address to a `DatagramPacket`.
- `void setData(byte[] buffer)`—assigns a new data buffer to the `DatagramPacket`. Remember to make the buffer long enough, to prevent data loss.
- `void setLength(int length)`—assigns a new length to the `DatagramPacket`. Remember that the length must be less than or equal to the maximum size of the data buffer, or an `IllegalArgumentException` will be thrown. When sending a smaller amount of data, you can adjust the length to fit—you do not need to resize the data buffer.
- `void setPort(int port)`—assigns a new destination port to a `DatagramPacket`.

5.3 DatagramSocket Class

The `DatagramSocket` class provides access to a UDP socket, which allows UDP packets to be sent and received. A `DatagramPacket` is used to represent a UDP packet, and must be created prior to receiving any packets. The same `DatagramSocket` can be used to receive packets as well as to send them. However, read operations are blocking, meaning that the application will continue to wait until a packet arrives. Since UDP packets do not guarantee delivery, this can cause an application to stall if the sender does not resubmit packets. You can use multiple threads of execution, discussed in [Chapter 7](#), or as of JDK1.1, you can use nonblocking I/O to avoid this problem, as shown in the `EchoClient` example discussed in [Section 5.7.2](#).

5.3.1 Creating a DatagramSocket

A `DatagramSocket` can be used to both send and receive packets. Each `DatagramSocket` binds to a port on the local machine, which is used for addressing packets. The port number need not match the port number of the remote machine, but if the application is a UDP server, it will usually choose a specific port number. If the `DatagramSocket` is intended to be a client, and doesn't need to bind to a specific port number, a blank constructor can be specified.

Constructors

To create a client `DatagramSocket`, the following constructor is used: `DatagramSocket()` throws `java.net.SocketException`. To create a server `DatagramSocket`, the following constructor is used, which takes as a parameter the port to which the UDP service will be bound: `DatagramSocket(int port)` throws `java.net.SocketException`.

Although rarely used, there is a third constructor for `DatagramSocket`, introduced in JDK1.1. If a machine is known by several IP addresses (referred to as multihomed), you can specify the IP address and port to which a UDP service should be bound. It takes as parameters the port to which the UDP service will be bound, as well as the `InetAddress` of the service. This constructor is: `DatagramSocket(int port, InetAddress addr)` throws `java.net.SocketException`.

5.3.2 Using a DatagramSocket

`DatagramSocket` is used to receive incoming UDP packets and to send outgoing UDP packets. It provides methods to send and receive packets, as well as to specify a timeout value when nonblocking I/O is being used, to inspect and modify maximum UDP packet sizes, and to close the socket.

Methods

- `void close()` — closes a socket, and unbinds it from the local port.
- `void connect(InetAddress remote_addr int remote_port)` — restricts access to the specified remote address and port. The designation is a misnomer, as UDP doesn't actually create a "connection" between one machine and another. However, if this method is used, it causes exceptions to be thrown if an attempt is made to send packets to, or read packets from, any other host and port than those specified.
- `void disconnect()` — disconnects the `DatagramSocket` and removes any restrictions imposed on it by an earlier connect operation.
- `InetAddress getInetAddress()` — returns the remote address to which the socket is connected, or null if no such connection exists.
- `int getPort()` — returns the remote port to which the socket is connected, or -1 if no such connection exists.
- `InetAddress getLocalAddress()` — returns the local address to which the socket is bound.
- `int getLocalPort()` — returns the local port to which the socket is bound.
- `int getReceiveBufferSize()` throws `java.net.SocketException` — returns the maximum buffer size used for incoming UDP packets.
- `int getSendBufferSize()` throws `java.net.SocketException` — returns the maximum buffer size used for outgoing UDP packets.
- `int getSoTimeout()` throws `java.net.SocketException` — returns the value of the timeout socket option. This value is used to determine the number of milliseconds a read operation will block before throwing a `java.io.InterruptedIOException`. By default, this value will be zero, indicating that blocking I/O will be used.
- `void receive(DatagramPacket packet)` throws `java.io.IOException` — reads a UDP packet and stores the contents in the specified packet. The address and port

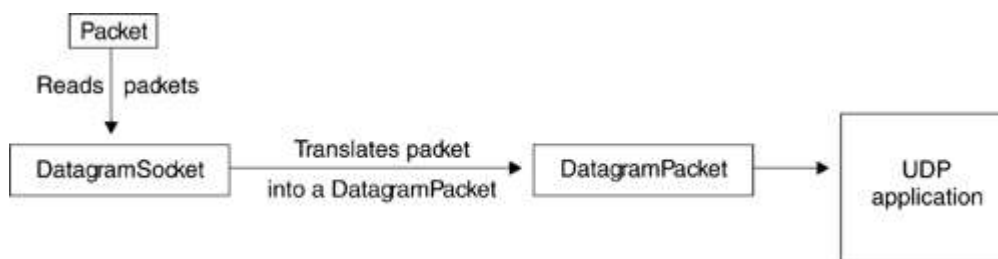
fields of the packet will be overwritten with the sender address and port fields, and the length field of the packet will contain the length of the original packet, which can be less than the size of the packet's byte-array. If a timeout value hasn't been specified by using `DatagramSocket.setSoTimeout(int duration)`, this method will block indefinitely. If a timeout value has been specified, a `java.io.InterruptedIOException` will be thrown if the time is exceeded.

- `void send(DatagramPacket packet)` throws `java.io.IOException`—sends a UDP packet, represented by the specified packet parameter.
- `void setReceiveBufferSize(int length)` throws `java.net.SocketException`—sets the maximum buffer size used for incoming UDP packets. Whether the specified length will be adhered to is dependent on the operating system.
- `void setSendBufferSize(int length)` throws `java.net.SocketException`—sets the maximum buffer size used for outgoing UDP packets. Whether the specified length will be adhered to is dependent on the operating system.
- `void setSoTimeout(int duration)` throws `java.net.SocketException`—sets the value of the timeout socket option. This value is the number of milliseconds a read operation will block before throwing a `java.io.InterruptedIOException`.

5.4 Listening for UDP Packets

Before an application can read UDP packets sent to it by remote machines, it must bind a socket to a local UDP port using `DatagramSocket`, and create a `DatagramPacket` that will act as a container for the UDP packet's data. [Figure 5-3](#) shows the relationship between a UDP packet, the various Java classes used to process it, and the actual application.

Figure 5-3. UDP packets are received by a `DatagramSocket` and translated into a `DatagramPacket` object.



When an application wishes to read UDP packets, it calls the `DatagramSocket.receive` method, which copies a UDP packet into the specified `DatagramPacket`. The contents of the `DatagramPacket` are processed, and the process is repeated as needed.

The following code snippet illustrates this process:

```

DatagramPacket packet = new DatagramPacket (new byte[256], 256);
DatagramSocket socket = new DatagramSocket(2000);
boolean finished = false;

while (! finished )
{
    socket.receive (packet);
}
  
```

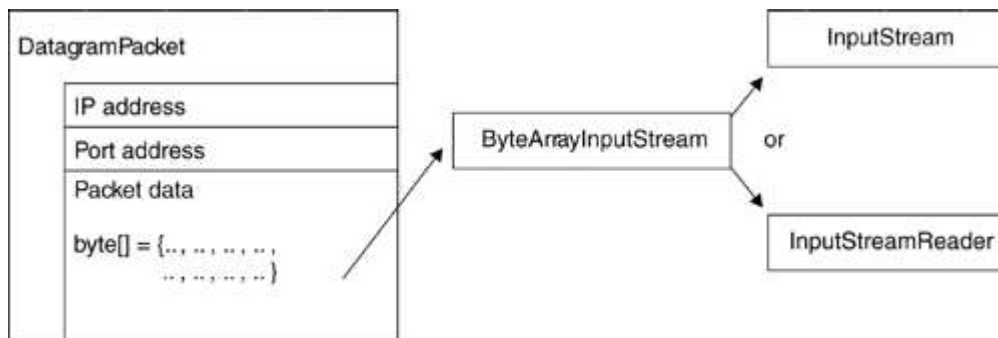
```

        // process the packet
    }
    socket.close();

```

When processing the packet, the application must work directly with an array of bytes. If, however, your application is better suited to reading text, you can use classes from the Java I/O package to convert between a byte array and another type of stream or reader. By hooking a `ByteArrayInputStream` to the contents of a datagram and then to another type of `InputStream` or an `InputStreamReader`, you can access the contents of UDP packets relatively easily (see [Figure 5-4](#)). Many developers prefer to use Java I/O streams to process data, using a `DataInputStream` or a `BufferedReader` to access the contents of byte arrays.

Figure 5-4. Reading from a UDP packet is simplified by applying input streams.



For example, to hook up a `DataInputStream` to the contents of a `DatagramPacket`, the following code can be used:

```

ByteArrayInputStream bin = new ByteArrayInputStream(
    packet.getData() );
DataInputStream din = new DataInputStream (bin);

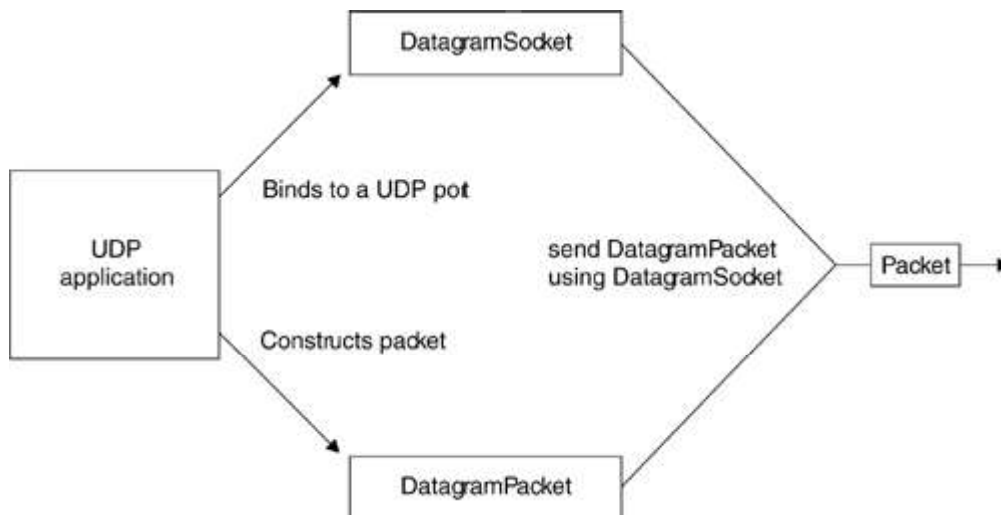
// Read the contents of the UDP packet
.....

```

5.5 Sending UDP packets

The same interface (`DatagramSocket`) employed to receive UDP packets is also used to send them. When sending a packet, the application must create a `DatagramPacket`, set the address and port information, and write the data intended for transmission to its byte array. If replying to a received packet, the address and port information will already be stored, and only the data need be overwritten. Once the packet is ready for transmission, the `send` method of `DatagramSocket` is invoked, and a UDP packet is sent (see [Figure 5-5](#)).

Figure 5-5. Packets are sent using a `DatagramSocket`.



The following code snippet illustrates this process:

```

DatagramSocket socket = new DatagramSocket(2000);
DatagramPacket packet = new DatagramPacket (new byte[256], 256);

packet.setAddress ( InetAddress.getByName ( somehost ) );
packet.setPort ( 2000 );

boolean finished = false;

while !finished )
{
    // Write data to packet buffer
    .....
    socket.send (packet);

    // Do something else, like read other packets, or check to
    // see if no more packets to send
    .....
}

socket.close();

```

5.6 User Datagram Protocol Example

To demonstrate how UDP packets are sent and received, we'll compile and run two small examples. The first will bind to a local port, read a packet, and display its contents and addressing information. The second example will send the packet read by the first.

Code for PacketReceiveDemo

```

import java.net.*;
import java.io.*;

// Chapter 5, Listing 1
public class PacketReceiveDemo
{
    public static void main (String args[])

```

```

{
    try
    {
        System.out.println ("Binding to local port
        2000");

        // Create a datagram socket, bound to the
        // specific port 2000
        DatagramSocket socket = new
        DatagramSocket(2000);

        System.out.println ("Bound to local port "
        + socket.getLocalPort());

        // Create a datagram packet, containing a
        // maximum buffer of 256 bytes
        DatagramPacket packet = new
        DatagramPacket( new byte[256], 256 );

        // Receive a packet - remember by default
        //this is a blocking operation
        socket.receive(packet);

        System.out.println ("Packet received!");

        // Display packet information
        InetAddress remote_addr = packet.getAddress();
        System.out.println ("Sent by : " +
remote_addr.getHostAddress() );
        System.out.println ("Sent from: " +
packet.getPort());

        // Display packet contents, by reading
        // from byte array
        ByteArrayInputStream bin = new
        ByteArrayInputStream (packet.getData());

        // Display only up to the length of the
        // original UDP packet
        for (int i=0; i < packet.getLength(); i++)
        {
            int data = bin.read();
            if (data == -1)
                break;
            else
                System.out.print ( (char)
                data) ;
        }

        socket.close();
    }
    catch (IOException ioe)
    {
        System.err.println ("Error - " + ioe);
    }
}
}

```

How PacketReceiveDemo Works

Most of the code is self-explanatory, or is similar to code snippets shown earlier. However, readers may benefit from a closer examination.

The application starts by binding to a specific port, 2000. Applications offering a service generally bind to a specific port. When acting as a receiver, your application should choose a specific port number, so that a sender can send UDP packets to this port. Next, the application prepares a `DatagramPacket` for storing UDP packets, and creates a new buffer for storing packet data.

```
// Create a datagram socket, bound to the specific port 2000
DatagramSocket socket = new DatagramSocket(2000);

System.out.println ("Bound to local port " +
socket.getLocalPort());

// Create a datagram packet, containing a maximum buffer of 256
// bytes
DatagramPacket packet = new DatagramPacket( new byte[256], 256
);
```

Now the application is ready to read a packet. The read operation is blocking, so until a packet arrives, the server will wait. When a packet is successfully delivered to the application, the addressing information for the packet is displayed so that it can be determined where it came from.

```
// Receive a packet - remember by default this is a blocking
// operation
socket.receive(packet);

// Display packet information
InetAddress remote_addr = packet.getAddress();
System.out.println ("Sent by : " + remote_addr.getHostAddress() );
System.out.println ("Send from: " + packet.getPort());
```

To provide easy access to the contents of the UDP packet, the application uses a `ByteArrayInputStream` to read from the packet. Reading one character at a time, the program displays the contents of the packet and then finishes. Note that Unicode characters, which are represented by more than just a single byte, cannot be written out in this fashion (readers and writers would be more appropriate if internationalization support is required).

```
// Display packet contents, by reading from byte array
ByteArrayInputStream bin = new ByteArrayInputStream
(packet.getData());

// Display only up to the length of the original UDP packet
for (int i=0; i < packet.getLength(); i++)
{
    int data = bin.read();
    if (data == -1)
        break;
    else
        System.out.print ( (char) data) ;
}
```

Code for PacketSendDemo

```
import java.net.*;
import java.io.*;

// Chapter 5, Listing 2
```

```

public class PacketSendDemo
{
    public static void main (String args[])
    {
        int argc = args.length;

        // Check for valid number of parameters
        if (argc != 1)
        {
            System.out.println ("Syntax :");
            System.out.println ("java PacketSendDemo
hostname");
            return;
        }

        String hostname = args[0];

        try
        {
            System.out.println ("Binding to a local port");
            // Create a datagram socket, bound to any
available
            // local port
            DatagramSocket socket = new
            DatagramSocket();

            System.out.println ("Bound to local port "
+ socket.getLocalPort());
            // Create a message to send using a UDP packet
            ByteArrayOutputStream bout = new
            ByteArrayOutputStream();
            PrintStream pout = new PrintStream (bout);
            pout.print ("Greetings!");

            // Get the contents of our message as an array
            // of bytes
            byte[] barray = bout.toByteArray();

            // Create a datagram packet, containing our byte
            // array

            DatagramPacket packet = new
            DatagramPacket( barray, barray.length );

            System.out.println ("Looking up hostname "
+ hostname );

            // Lookup the specified hostname, and get an
            // InetAddress
            InetAddress remote_addr =
            InetAddress.getByName(hostname);

            System.out.println ("Hostname resolved as " +
remote_addr.getHostAddress());

            // Address packet to sender
            packet.setAddress (remote_addr);

            // Set port number to 2000
            packet.setPort (2000);

```

```

        // Send the packet - remember no guarantee of
delivery        socket.send(packet);

        System.out.println ("Packet sent!");
    }
    catch (UnknownHostException uhe)
    {
        System.err.println ("Can't find host " +
hostname);
    }
    catch (IOException ioe)
    {
        System.err.println ("Error - " + ioe);
    }
}
}

```

How PacketSendDemo Works

The second example uses UDP to talk to the first example. This example acts as the sender, dispatching a UDP packet to the receiver, which contains an ASCII text-greeting message. Though it uses some similar classes (`DatagramSocket`, `DatagramPacket`), they are employed in a slightly different way.

The application starts by binding a UDP socket to a local port, which will be used to send the data packet. Unlike the receiver demonstration, it doesn't matter which local port is being used. In fact, any free port is a candidate, and you may find that running the application several times will result in different port numbers. After binding to a port, the port number is displayed to demonstrate this.

```

// Create a datagram socket, bound to any available local port
DatagramSocket socket = new DatagramSocket();
System.out.println ("Bound to local port " +
socket.getLocalPort());

```

Before sending any data, we need to create a `DatagramPacket`. First, a `ByteArrayOutputStream` is used to create a sequence of bytes. Once this is complete, the array of bytes is passed to the `DatagramPacket` constructor.

```

// Create a message to send using a UDP packet
ByteArrayOutputStream bout = new ByteArrayOutputStream();
PrintStream pout = new PrintStream (bout);
pout.print ("Greetings!");

// Get the contents of our message as an array of bytes
byte[] barray = bout.toByteArray();

// Create a datagram packet, containing our byte array
DatagramPacket packet = new DatagramPacket( barray,
barray.length );

```

Now that the packet has some data, it needs to be correctly addressed. As with a postal message, if it lacks correct address information it cannot be delivered. We start by obtaining an `InetAddress` for the remote machine, and then display its IP address. This `InetAddress` is passed to the `setAddress` method of `DatagramPacket`, ensuring that it will arrive at the correct machine. However, we must go one step further and specify a port number. In this case, port 2000 is matched, as the receiver will be bound to that port.

```

System.out.println ("Looking up hostname " + hostname );

// Lookup the specified hostname, and get an InetAddress
InetAddress remote_addr = InetAddress.getByName(hostname);

System.out.println ("Hostname resolved as " +
remote_addr.getHostAddress());

// Address packet to sender
packet.setAddress (remote_addr);

// Set port number to 2000
packet.setPort      (2000);

```

The final step, after all this work, is to send the packet. This is the easiest step of all—simply invoke the send method of `DatagramSocket`. Again, remember: there is no guarantee of delivery, so it is possible for a packet to become lost in transit. A more robust application would try to read an acknowledgment and resend the message if it had become lost.

```

// Send the packet - remember no guarantee of delivery
socket.send(packet);

```

Running the UDP Examples

To run these examples, you'll need to open two console windows. The first application to be run is the receiver, which will wait for a UDP packet.

There are no parameters for the receiver, so to run it use:

```
java PacketReceiveDemo
```

In a second window, you then need to run the sender. This application could be run from any computer on a local network or the Internet (providing there isn't a firewall between the two hosts). If you'd like, you can also run it from the same machine. It takes a single parameter, the hostname of the remote machine:

```
java PacketSendDemo myhostname
```

NOTE



As mentioned in earlier chapters, you can use `localhost` as the hostname to refer to the local machine.

5.7 Building a UDP Client/Server

The previous example illustrates the technical details of how an individual packet may be sent and received. But applications need a series of packets, not just one. The next example shows how to build a UDP server, a long-running system that is capable of serving many requests during its lifetime. The type of service that is provided is an echo service, which echoes back the contents of

a packet. The echo service runs on a well-known port, port 7, and if it is known that a system has an echo server installed, the server may be accessed by clients to see if a system is up and running (similar to the ping application). The example below demonstrates how to write an echo client that will send packets to the server as well as read the results back.

NOTE



Some systems have an echo service already running in the background, and security restrictions may prevent a service from binding to a well-known port. If this is the case, you will need to change the port number to a different number (and in the case of security restrictions, you should select a number above port 1024) in both the client and the server. The same port number must be used for communication.

5.7.1 Building an Echo Service

The following example involves building an echo service, which transmits any packet it receives straight back to the sender. The code uses no new networking classes or methods, but employs a special technique. It loops continuously to serve one client after another. Though only one UDP packet will be processed at a time, the delay between receiving a packet and dispatching it again is negligible, resulting in the illusion of concurrent processing.

```
import java.net.*;
import java.io.*;

// Chapter 5, Listing 3
public class EchoServer
{
    // UDP port to which service is bound
    public static final int SERVICE_PORT = 7;

    // Max size of packet, large enough for almost any client
    public static final int BUFSIZE = 4096;

    // Socket used for reading and writing UDP packets
    private DatagramSocket socket;
    public EchoServer()
    {
        try
        {
            // Bind to the specified UDP port, to listen
            // for incoming data packets
            socket = new DatagramSocket( SERVICE_PORT );

            System.out.println ( "Server active on port " +
                                socket.getLocalPort() );
        }
        catch (Exception e)
        {
            System.err.println ( "Unable to bind port" );
        }
    }

    public void serviceClients()
    {
        // Create a buffer large enough for incoming packets
        byte[] buffer = new byte[BUFSIZE];
```

```

        for (;;)
        {
            try
            {
                // Create a DatagramPacket for reading
                // UDP packets
                DatagramPacket packet = new DatagramPacket
                    ( buffer, BUFSIZE );

                // Receive incoming packets
                socket.receive(packet);

                System.out.println ("Packet received from " +
                    packet.getAddress() + ":" +
                    packet.getPort() +
                    " of length " + packet.getLength() );

                // Echo the packet back - address and port
                // are already set for us !
                socket.send(packet);
            }
            catch (IOException ioe)
            {
                System.err.println ("Error : " + ioe);
            }
        }
    }

    public static void main(String args[])
    {
        EchoServer server = new EchoServer();
        server.serviceClients();
    }
}

```

5.7.2 Building an Echo Client

The following client can be used with the echo service and can easily be adapted to support other services. Repeated packets are sent to the echo service, and a timeout is caught to prevent the service from stalling if a packet becomes lost, and the client then waits to receive it. Remember that packet loss in an intranet environment is unlikely, but with slow network connections on the Internet it is quite possible.

```

import java.net.*;
import java.io.*;

// Chapter 5, Listing 4
public class EchoClient
{
    // UDP port to which service is bound
    public static final int SERVICE_PORT = 7;

    // Max size of packet
    public static final int BUFSIZE = 256;

    public static void main(String args[])
    {
        if (args.length != 1)
        {

```



```

        System.err.println ("Syntax - java EchoClient
hostname");
        return;
    }

    String hostname = args[0];

    // Get an InetAddress for the specified hostname
    InetAddress addr = null;
    try
    {
        // Resolve the hostname to an InetAddr
        addr = InetAddress.getByName(hostname);
    }
    catch (UnknownHostException uhe)
    {
        System.err.println ("Unable to resolve host");
        return;
    }

    try
    {
        // Bind to any free port
        DatagramSocket socket = new DatagramSocket();

        // Set a timeout value of two seconds
        socket.setSoTimeout (2 * 1000);

        for (int i = 1 ; i <= 10; i++)
        {
            // Copy some data to our packet
            String message = "Packet number " + i ;
            char[] cArray = message.toCharArray();
            byte[] sendbuf = new byte[cArray.length];

            for (int offset = 0; offset <
cArray.length ; offset++)
            {
                sendbuf[offset] = (byte)
cArray[offset];
            }

            // Create a packet to send to the UDP
            server
            DatagramPacket sendPacket = new
            DatagramPacket(sendbuf,
                cArray.length, addr, SERVICE_PORT);

            System.out.println ("Sending packet to " +
hostname);

            // Send the packet
            socket.send (sendPacket);

            System.out.print ("Waiting for packet.... ");

            // Create a small packet for receiving UDP
            packets

            byte[] recbuf = new byte[BUFSIZE];
            DatagramPacket receivePacket = new
            DatagramPacket(recbuf,

```

```

        BUFSIZE);

        // Declare a timeout flag
        boolean timeout = false;

        // Catch any InterruptedException that is
thrown
        // while waiting to receive a UDP packet
        try
        {
            socket.receive (receivePacket);
        }
        catch (InterruptedException ioe)
        {
            timeout = true;
        }

        if (!timeout)
        {
            System.out.println ("packet received!");
            System.out.println ("Details : " +
                receivePacket.getAddress());

            // Obtain a byte input stream to read the
            // UDP packet
            ByteArrayInputStream bin = new
            ByteArrayInputStream (
                receivePacket.getData(), 0,
                receivePacket.getLength() );

            // Connect a reader for easier access
            BufferedReader reader = new
            BufferedReader (
                new InputStreamReader ( bin ) );

            // Loop indefinitely
            for (;;)
            {
                String line = reader.readLine();

                // Check for end of data
                if (line == null)
                    break;
                else
                    System.out.println (line);
            }
        }
        else
        {
            System.out.println ("packet lost!");
        }

        // Sleep for a second, to allow user to see
packet
        try
        {
            Thread.sleep(1000);
        } catch (InterruptedException ie) { }
    }
}
catch (IOException ioe)

```

```

        {
            System.err.println ("Socket error " + ioe);
        }
    }
}

```

5.7.3 Running the Echo Client and Server

Before clients can send requests, the echo server must be active. Otherwise, UDP packets will be sent and then ignored, as there is no program running to read them. However, the client will not stall if a response is not sent back; it will send packets again after two seconds of waiting. This is important, as servers may be inactive or packets may become lost in transmission.

To run the echo server, type the following:

```
java EchoServer
```

To run the echo client (either on the same machine as the server or a different one), type the following:

```
java EchoClient hostname
```

where hostname (or localhost, if running locally) is the location of the echo service.

NOTE



Remember that on Unix systems, you must have root privileges to bind to a port under 1024. You can change the service port to a higher number, but this must be done in both the client and the server in order to recompile.

5.8 Additional Information on UDP

While the UDP is sometimes the best alternative for certain classes of applications, because of its unique properties, it does present some challenges to developers. These challenges can be met, however, by structuring data transmission to overcome the limitations of UDP. Below we examine these limitations and how they may be overcome.

5.8.1 Lack of Guaranteed Delivery

Packets sent via UDP may become lost in transit—each additional hop between one router and another introduces more delays and increases the likelihood that a packet may be discarded when its TTL reaches zero. Furthermore, UDP packets can become damaged or lost if the physical network connection they are being routed through goes down. Since Internet packets are being transmitted across a public network, composed of a diverse range of network infrastructures, it is likely that packets will become lost at some point in a connection.

Of course, in some applications the loss of individual packets may not have a noticeable effect. For example, a video stream might lose a few frames of picture, but provided that most of the frames arrive, the loss is bearable. However, if a file is being transferred, then the file contents

will become garbled, and the loss of packets becomes unacceptable. If guaranteed delivery is required, the best alternative is to avoid packet-based communication altogether and use a more suitable transport mechanism like the Transmission Control Protocol (TCP), discussed in [Chapter 6](#). Nonetheless, if the use of UDP is called for, one solution is for the party receiving packets to send an acknowledgment packet (also referred to as an ACK) back to the sender. The absence of an ACK indicates that a packet was lost and should be retransmitted.

NOTE



Some transport systems send back an ACK for individual packets or for a range of packets. Although it does add additional complexity, acknowledgment of a range of packets makes for a more efficient use of bandwidth. Some systems also use a negative-acknowledgment packet (NAK) to indicate that a specific packet was lost, which triggers immediate retransmission of that packet.

5.8.2 Lack of Guaranteed Packet Sequencing

Applications that require sequential access to data (and let's face it, that amounts to most software) should include a sequence number in the contents of a datagram packet. If a packet arrives out of order, it can be buffered until the earlier packets have caught up. Sequencing adds a small amount of complexity, but does make a system more reliable—you always know which packet you're dealing with! Duplicate packets must be discarded, and missing packets (due to a lack of guaranteed delivery) requested again.

5.8.3 Lack of Flow Control

The face of the Internet is changing rapidly, as network connections move from dial-up and wireless modems to broadband communication through cable modems and ISDN lines. Some systems can handle a large amount of data, whereas others still have extremely limited bandwidth. To avoid flooding a system with more data than it can handle, the technique of flow control is used. Flow control places a limit on how much data is sent, and helps to prevent systems from becoming overloaded (and bandwidth from being wasted). Imagine flow control as a water limiter that restricts the amount of liquid flowing through a showerhead.

There are many flow-control techniques, ranging from limiting the number of packets sent per second to limiting the number of packets that have not yet been acknowledged. The settings for the former are hard to determine, as they vary depending on the receiver. The latter is probably the best choice—by limiting the number of unacknowledged packets, control is placed in the hands of the receiver. If the receiver can get and respond to packets quickly, then more packets will be acknowledged and more will be sent. If packets are flooding the network, fewer responses will come back and a throttle is placed on the flow. Some systems may also elect to use a variable flow limit, which can be customized to take into account the length of time for acknowledgments to come back. Since UDP doesn't offer direct control over flow control, for large-scale applications it may be appropriate to limit the number of packets sent to a host (for example, n packets per second, where n is a number suitable for the transmission line and speed of the recipient machine).

5.9 Summary

The User Datagram Protocol (UDP) provides a connectionless transport mechanism that allows packets of data to be sent to remote machines. UDP generates less overhead than other transport

protocols and is highly desirable for real-time broadcasting of multimedia such as audio, video, and network gaming data. However, due to its lack of guaranteed delivery and sequencing, it creates additional work for developers, and thus the pros and cons of its use must be considered on a case-by-case basis.

Java supports UDP in the form of the `DatagramSocket` and `DatagramPacket` classes. In order to send or receive packets, an application must bind to a local port using `DatagramSocket`. The application creates a `DatagramPacket` with a sufficient buffer to store incoming or outgoing packets, and can then send and receive packets as required.

Chapter Highlights

In this chapter, you have learned:

- How to bind to a local port using `DatagramSocket`
- How to create a `DatagramPacket`
- How to read from, and write to, a `DatagramPacket` using `ByteArrayInputStream` and `ByteArrayOutputStream`
- How to listen for UDP packets
- How to send UDP packets
- How to create a UDP server and a UDP client